UNIVERSITY OF CEBU - MAIN CAMPUS
COLLEGE OF COMPUTER STUDIES
SANGCIANGKO ST, CEBU CITY, 6000 CEBU

Fair Group Assignment

"Equally Balancing Teams, Maximizing Potential"

Submitted by:

Villegas, Roginand

BSCS 2A

Submitted to:

Mr. Christian C. Barral

May 17, 2025

Problem Statement

In voluntary group settings within educational environments, students often self-select into groups based on personal preferences. This can lead to imbalances in academic performance across groups. Irregularities such as differing class schedules and social dynamics further complicate the formation of equitable groups. As a result, some groups may have a concentration of high-performing students, while others may consist mostly of lower-performing members.

Even random sampling, though seemingly impartial, can still produce uneven group compositions—capable students might end up in the same group by chance, widening performance disparities. These imbalances create unequal group dynamics and can foster an unfair learning environment.

This algorithm aims to address that issue. It automates group assignment with the specific goal of balancing groups based on student grades (of a particular course or subject), promoting fairness and effective collaboration.

Inputs:

- A set of students, each with a name and a grade (float value, 1.0 ≤ grade ≤ 5.0).
- An integer specifying the number of groups (groupAmount).
- Example input: 60 students with grades ranging from 1.0 to 4.0, to be divided into 8 groups.

Expected Output:

- A list of groups, each containing a subset of students, with name and average grades as close as possible.
- Example output: 8 groups with average grades approximately equal, e.g., 2.45, 2.46, etc.

Constraints:

- Grades must be valid (1.0 ≤ grade ≤ 5.0).
- The number of students must be at least equal to the number of groups.
- The algorithm minimizes differences in group average grades while maintaining

---

Algorithm 1: Round-Robin Assignment

- **Name:** Cyclic Sorted Distribution
- **Purpose:** Distribute students evenly across groups without dynamic average optimization.
- **How It Works:**
  - Students are sorted by grade.
  - Assigned cyclically (e.g., Group 1 → Group 2 → ... → Group *n* → Group 1 → ...).
- **Strengths:**
  - Simpler and faster (O(n) time).
  - Guarantees even spread of high/low performers.
- **Weaknesses:**
  - May produce suboptimal initial averages (unlike greedy partitioning).
  - Requires post-assignment balancing (e.g., swaps).

Algorithm 2: Greedy Partitioning

- **Name:** Greedy Partitioning by Grade
- **Purpose:** Partitions students into groups by greedily selecting based on grade order.
- **How It Works:** Students are sorted by grade, then assigned to groups in a way that greedily balances the groups, such as using a round-robin approach.
- **Strengths:**
  - Creates a more balanced starting point compared to random assignment.
  - Efficient for initial distribution.
- **Weaknesses:**
  - May not fully optimize balance without further adjustments.
  - Greedy choices can overlook better overall solutions.

Algorithm 3: Divide and Conquer

- **Name:** Divide and Conquer Balancing
- **Purpose:** Recursively splits the problem into smaller subproblems to achieve balance efficiently.
- **How It Works:** The algorithm divides the groups into smaller subsets, balances each subset individually, and then merges them while ensuring balance across the merged groups. This continues until the entire set is balanced.
- **Strengths:**
  - Reduces complexity by breaking down the problem, making it scalable for larger datasets.
  - Ensures thorough balancing by addressing smaller, manageable parts before combining them.
- **Weaknesses:**
  - Requires careful handling of recursion to avoid inefficiencies.

    - May need additional mechanisms to handle edge cases, such as uneven group sizes.

Why Combining These Approaches Is Better for This Algorithm

This algorithm combines the strengths of these two approaches to create a fair and efficient group assignment solution:

- **Initial Sorted Assignment (from Greedy Partitioning):** Starting with a sorted list of students and distributing them evenly across groups provides a strong, balanced foundation. This avoids the uneven starting points that can occur with random assignment.
- **Round-Robin Assignment:** By cycling through groups when assigning sorted students, the algorithm achieves immediate balance in group composition. This approach maintains consistent group sizes and guarantees fair representation of all performance levels in each group, creating an optimal starting point for further refinement.
- **Divide and Conquer Balancing:** By breaking the problem into smaller subsets, balancing them individually, and then merging them carefully, the algorithm ensures fairness across all groups without needing to compare every pair of groups at once. This makes it practical even for large numbers of students.

Together, these methods produce a result that's both fair and manageable. The sorted start gives a solid base, swapping fine-tunes it, and divide-and-conquer keeps the process efficient and scalable. No single approach could achieve this combination of fairness and practicality as effectively.

Algorithm Design

Overview / Strategy

The Fair Group Assignment algorithm employs a three-phase approach to achieve balanced groups:

1. **Sorting Students:** Orders students by grade to enable a fair initial distribution.
2. **Initial Assignment:** Uses a round-robin method to evenly distribute students across groups.
3. **Balancing Phase:** Applies a divide-and-conquer strategy to recursively refine group averages through targeted swaps.

This design integrates the simplicity of greedy partition and round-robin distribution with the power of divide-and-conquer, a paradigm widely recognized for its efficiency in solving complex problems.

Step-by-Step Description

1. **Initialize Student Set:**
    - Store students in a sorted set by grade (ascending). Ties are resolved alphabetically by name.
    - Filter out invalid grades (e.g., > 5.0 or < 1.0) to ensure data integrity.
    - Example: For 60 students, the sorted list might range from 1.0 to 4.0.
2. **Initial Assignment:**
    - Compute the base group size: total students / groupAmount.
    - Distribute remainder students (if any) to initial groups, e.g., some groups get baseSize + 1.
    - Assign students round-robin from the sorted list, ensuring a balanced spread of grades.
    - Example: For 60 students and 8 groups, each group gets 7 or 8 students.
3. **Balancing Phase (Divide-and-Conquer):**
    - **Divide:** Recursively split the group list into left and right halves until each subset contains one group.
    - **Conquer:** For each subset, identify groups with the highest and lowest average grades.
    - **Merge and Balance:** Swap students between these groups if it reduces the average difference. Repeat until no further improvements are possible within the subset.
    - Recursion continues across all subsets, merging results to balance the entire set.
    - Example: With 8 groups, the algorithm splits into 4 vs. 4, then 2 vs. 2, and so on, balancing at each level.
4. **Output Groups:**
    - Compute and display each group's student list and average grade post-balancing.
    - Example output: 8 groups with averages like 2.45, 2.46, etc., with minimal differences.

Rules for Balancing

- Swaps occur only if they decrease the difference between group averages.
- Group sizes remain fixed as per the initial assignment.
- The process stops when no beneficial swaps remain, ensuring an optimal solution.

Divide-and-Conquer in Depth

The divide-and-conquer strategy is the cornerstone of the balancing phase, inspired by algorithms like merge sort. Here's a detailed breakdown:

- **Divide:** The list of groups is halved recursively (e.g., 8 groups → 4 and 4 → 2 and 2 → 1 ad 1). This reduces the problem size at each step.
- **Conquer:** For each subset, the algorithm:
    - Calculates the average grade of each group.
    - Identifies the maximum and minimum average groups.
    - Tests possible swaps between these groups.
- **Merge and Balance:** During merging, swaps are executed if they improve balance. This process repeats within each merged subset until the entire set is optimized.
- **Balance (Greedy Swapping)**: Perform pairwise swaps between the identified groups only if they reduce the average difference (improveBySwapping)
    - Swaps are greedy: the best improvement is chosen first (not exhaustive).

Pseudocode

(Data is automatically sorted due to the TreeSet data structure)

- **Function: Add a student to the group**
    - If grade is > 5.0 or grade is < 1.0, print "Skipping student [name] with invalid grade" and return
    - Else, add the student to the studentWithGrade set
- **Function: Print all students**
    - Print student name and grade
- **Function: Assign Group**
    - Determine the total number of students
        - If total students is less than the required number of groups, print "Not enough students to form groups" and return null
    - Initialize groups as a list of empty lists based on groupAmount
    - Create a deque sortedStudentsDeque from studentWithGrade for easy removal of students
    - Calculate the base number of students per group (baseStudentsPerGroup) and any remainder
    - Initialize groupSizes array where each group will get either baseStudentsPerGroup or one more student if there is a remainder
    - While sortedStudentsDeque is not empty:
        - For each group index:
            - If the group's size is less than the corresponding size in groupSizes:
            - Pop the first student from sortedStudentsDeque and add to the group

    - Call balanceGroupsDivideConquer(groups, 0, groupAmount - 1) to balance the groups
    - Print group assignments with average grades
    - Return a list that contains a list of group
- **Function: Calculate Group Average**
    - Add all the grades of a group
    - Return the sum divided by the size of the group
- **Function: Balance groups using divide and conquer**
    - If the left index is greater than or equal to the right index, return
    - Calculate the midpoint (mid) between left and right
    - Recursively call balanceGroupsDivideConquer(groups, left, mid) and balanceGroupsDivideConquer(groups, mid + 1, right)
    - After recursion, call mergeAndBalance(groups, left, right) to balance the groups
- **Function: Merge and Balance**
    - Start a loop that will repeat as long as any swap is made
    - Inside the loop, assume that no swap will happen yet
    - Look at each group between the given left and right boundaries
    - For each group, calculate the average GPA of its students
    - Identify the group that has the highest average GPA, Save its index
    - Identify the group that has the lowest average GPA, Save its index
    - Once you know the highest and lowest GPA, call the improveBySwapping function
    - if improveBySwapping is True
        - swap is made, so it will loop again
    - if improveBySwapping is False
        - End the loop
- **Function: Swap students between two groups to improve balance**
    - For each student in group 1 and group 2:
        - Calculate the potential new averages if a swap occurred
        - If the swap improves the balance (reduces the difference in averages):
            - Perform the swap
            - Return true (indicating that a swap was made)
    - Return false if no improvement is found

Test Cases

Test Case 1: Standard Distribution

- **Input:**

```java
FairGroupAssignment fga = new FairGroupAssignment();
fga.add(new Student(name:"Zara Hill", grade:1.05));
fga.add(new Student(name:"Yara Knight", grade:1.10));
fga.add(new Student(name:"Winston Browne", grade:1.30));
fga.add(new Student(name:"Vera King", grade:1.40));
fga.add(new Student(name:"Umar Harris", grade:1.50));
fga.add(new Student(name:"Tina Lee", grade:1.60));
fga.add(new Student(name:"Sam Lee", grade:1.70));
fga.add(new Student(name:"Rachel Wong", grade:1.80));
fga.add(new Student(name:"Quinn Park", grade:1.90));
fga.add(new Student(name:"Paulina Carter", grade:2.00));
fga.add(new Student(name:"Olga Allen", grade:2.10));
fga.add(new Student(name:"Nancy Cruz", grade:2.20));
fga.add(new Student(name:"Mila Foster", grade:2.30));
fga.add(new Student(name:"Leo Martin", grade:2.40));
fga.add(new Student(name:"Viso Vaughn", grade:2.30));


fga.assignGroup(groupAmount:3);
```

- **Output:**

```
        --- Group Assignments ---
Group 1 (Avg: 1.78) - 5 students:
  - Yara Knight (1.1)
  - Umar Harris (1.5)
  - Rachel Wong (1.8)
  - Nancy Cruz (2.2)
  - Mila Foster (2.3)

Group 2 (Avg: 1.77) - 5 students:
  - Zara Hill (1.05)
  - Vera King (1.4)
  - Sam Lee (1.7)
  - Viso Vaughn (2.3)
  - Leo Martin (2.4)

Group 3 (Avg: 1.78) - 5 students:
  - Winston Browne (1.3)
  - Tina Lee (1.6)
  - Quinn Park (1.9)
  - Paulina Carter (2.0)
  - Olga Allen (2.1)
```

- **Explanation:** The students are automatically sorted by grade using a **TreeSet**, ensuring they're in order before being assigned to groups. The distribution follows a round-robin method, where students are assigned cyclically across the groups. After the initial assignment, a divide-and-conquer optimization phase balances the groups by swapping students between them, ensuring the final groups have balanced average grades.

Step 1: Initialize Groups

The algorithm begins by creating a list of empty sublists, where each sublist represents a group. The number of sublists corresponds to the total number of groups.

Example for 3 groups:

groups = [[], [], []]

Step 2: Determine Base Students per Group & Remainder

To distribute students evenly:

Calculate the base number of students per group using integer division: students // groups.

Calculate the remainder using the modulo operator: students % groups. The remainder indicates how many groups will receive one extra student.

Example: For 15 students and 3 groups:

Base students per group = 15 // 3 = 5

Remainder = 15 % 3 = 0 (each group gets exactly 5 students)

Step 3: Round-Robin Assignment

Students are assigned to groups in a round-robin fashion. Starting with the first group, each student is placed in the next group in sequence. After assigning a student to the last group, the algorithm loops back to the first group. If there is a remainder, the first remaining groups receive one additional student.

Example Assignment:

Given 15 students with their grades, the round-robin assignment yields:

Group 1: Zara Hill (1.05), Vera King (1.40), Sam Lee (1.70), Paulina Carter (2.00), Mila Foster (2.30)

Group 2: Yara Knight (1.10), Umar Harris (1.50), Rachel Wong (1.80), Olga Allen (2.10), Leo Martin (2.40)

Group 3: Winston Browne (1.30), Tina Lee (1.60), Quinn Park (1.90), Nancy Cruz (2.20), Viso Vaughn (2.30)

Step 4: Calculate Initial Group Averages

After the round-robin assignment, compute the average grade for each group to assess balance.

Group Averages:

Group 1: (1.05 + 1.40 + 1.70 + 2.00 + 2.30) / 5 = 1.69

Group 2: (1.10 + 1.50 + 1.80 + 2.10 + 2.40) / 5 = 1.68

Group 3: (1.30 + 1.60 + 1.90 + 2.20 + 2.30) / 5 = 1.96

The initial averages show some imbalance, with Group 3 having a higher average (1.96) compared to Group 1 (1.69) and Group 2 (1.68).

Step 5: Balancing the Groups (Divide-and-Conquer)

To achieve balanced groups, a divide-and-conquer approach is applied. This phase recursively splits the list of groups, balances subsets by swapping students, and merges the subsets while continuing to refine the balance.

Divide

The list of groups is recursively split into two halves until each subset contains a single group.

Example for 3 groups:

Initial list: [Group 1, Group 2, Group 3]

Split into: [Group 1, Group 2] and [Group 3]

Conquer

For each pair of groups or subsets:

Calculate the average grade for each group or subset.

Identify the group (or subset) with the highest average and the one with the lowest average.

Attempt to swap students between these groups to reduce the difference in their averages. A swap is executed only if it decreases the average difference.

Example Swap:

Consider the initial groups:

Group 2 (lowest average: 1.68) has students: Yara Knight (1.10), Umar Harris (1.50), Rachel Wong (1.80), Olga Allen (2.10), Leo Martin (2.40)

Group 3 (highest average: 1.96) has students: Winston Browne (1.30), Tina Lee (1.60), Quinn Park (1.90), Nancy Cruz (2.20), Viso Vaughn (2.30)

Swapping Rachel Wong (1.80) from Group 2 with Winston Browne (1.30) from Group 3:

New Group 2: Yara Knight (1.10), Umar Harris (1.50), Winston Browne (1.30), Olga Allen (2.10), Leo Martin (2.40)

New average: (1.10 + 1.50 + 1.30 + 2.10 + 2.40) / 5 = 1.68 + (1.30 - 1.80) / 5 = 1.58

New Group 3: Rachel Wong (1.80), Tina Lee (1.60), Quinn Park (1.90), Nancy Cruz (2.20), Viso Vaughn (2.30)

New average: (1.80 + 1.60 + 1.90 + 2.20 + 2.30) / 5 = 1.96 + (1.80 - 1.30) / 5 = 2.06

The difference in averages increases (from 1.96 - 1.68 = 0.28 to 2.06 - 1.58 = 0.48), so this swap is not beneficial. The algorithm tries other swaps to find a better balance.

Merge

After balancing the subsets, they are merged back into a single list, and the conquer step is repeated on the merged list to further refine the balance.

Example Merge:

After balancing [Group 1, Group 2] and [Group 3], merge back to [Group 1, Group 2, Group 3].

Re-evaluate averages and attempt swaps across all groups.

Hypothetical Beneficial Swap:

Suppose a swap between Group 1 and Group 3:

Swap Paulina Carter (2.00) from Group 1 with Winston Browne (1.30) from Group 3.

New Group 1: Zara Hill (1.05), Vera King (1.40), Sam Lee (1.70), Winston Browne (1.30), Mila Foster (2.30)

New average: (1.05 + 1.40 + 1.70 + 1.30 + 2.30) / 5 = 1.69 + (1.30 - 2.00) / 5 = 1.55

New Group 3: Paulina Carter (2.00), Tina Lee (1.60), Quinn Park (1.90), Nancy Cruz (2.20), Viso Vaughn (2.30)

New average: (2.00 + 1.60 + 1.90 + 2.20 + 2.30) / 5 = 1.96 + (2.00 - 1.30) / 5 = 2.10

This swap is also not ideal, as it increases imbalance. The algorithm continues testing swaps recursively.

Step 6: Final Group Assignment

After multiple iterations of the divide-and-conquer phase, the algorithm converges on a balanced assignment. The final groups aim for minimal differences in average grades.

Final Groups:

Group 1: Yara Knight (1.10), Umar Harris (1.50), Rachel Wong (1.80), Nancy Cruz (2.20), Mila Foster (2.30)

Average: (1.10 + 1.50 + 1.80 + 2.20 + 2.30) / 5 = 1.78

Group 2: Zara Hill (1.05), Vera King (1.40), Sam Lee (1.70), Viso Vaughn (2.30), Leo Martin (2.40)

Average: (1.05 + 1.40 + 1.70 + 2.30 + 2.40) / 5 = 1.77

Group 3: Winston Browne (1.30), Tina Lee (1.60), Quinn Park (1.90), Paulina Carter (2.00), Olga Allen (2.10)

Average: (1.30 + 1.60 + 1.90 + 2.00 + 2.10) / 5 = 1.78

The final averages (1.78, 1.77, 1.78) are closely balanced, with a maximum difference of 0.01, indicating a successful balancing phase.

Test Case 2: Not Enough Students to Form Groups

- **Input:**

```java
FairGroupAssignment fga = new FairGroupAssignment();
fga.add(new Student(name:"Zara Hill", grade:1.05));
fga.add(new Student(name:"Yara Knight", grade:1.10));
fga.add(new Student(name:"Xander Evans", grade:1.20));
fga.add(new Student(name:"Winston Browne", grade:1.30));
fga.add(new Student(name:"Vera King", grade:1.40));

fga.assignGroup(groupAmount:6);
```

- **Output:**

```
Not enough students to form 6 groups (minimum 1 student per group).
```

- **Explanation:** Before distributing the students, the algorithm first checks the size of the TreeSet (which contains the sorted list of students). It compares the number of students to the requested groupAmount. If there are not enough students to form the specified number of groups (if the TreeSet size is less than groupAmount), the method will print an error message and return null, as it is impossible to create the requested number of groups with the available students.

Test Case 3: Uneven Students to the number of groups

- **Input:**

```
FairGroupAssignment fga = new FairGroupAssignment();
fga.add(new Student(name:"Zara Hill", grade:1.05));
fga.add(new Student(name:"Yara Knight", grade:1.10));
fga.add(new Student(name:"Xander Evans", grade:1.20));
fga.add(new Student(name:"Winston Browne", grade:1.30));
fga.add(new Student(name:"Vera King", grade:1.40));
fga.add(new Student(name:"Umar Harris", grade:1.50));
fga.add(new Student(name:"Tina Lee", grade:1.60));
fga.add(new Student(name:"Sam Lee", grade:1.70));
fga.add(new Student(name:"Rachel Wong", grade:1.80));
fga.add(new Student(name:"Quinn Park", grade:1.90));
fga.add(new Student(name:"Paulina Carter", grade:2.00));
fga.add(new Student(name:"Olga Allen", grade:2.10));
fga.add(new Student(name:"Nancy Cruz", grade:2.20));
fga.add(new Student(name:"Mila Foster", grade:2.30));
fga.add(new Student(name:"Leo Martin", grade:2.40));
fga.add(new Student(name:"Katherine Green", grade:2.50));
fga.add(new Student(name:"Jackie Cohen", grade:2.60));
fga.add(new Student(name:"Isaac Moore", grade:3.00));
fga.add(new Student(name:"Henry Rodriguez", grade:3.10));
fga.add(new Student(name:"George Brown", grade:3.50));
fga.add(new Student(name:"Roginand", grade:3.23));
fga.add(new Student(name:"John", grade:3.19));

fga.assignGroup(groupAmount:4);
```

- **Output:**

```
--- Group Assignments ---
Group 1 (Avg: 2.13) - 6 students:
  - Zara Hill (1.05)
  - Vera King (1.4)
  - Rachel Wong (1.8)
  - Mila Foster (2.3)
  - Isaac Moore (3.0)
  - Roginand (3.23)

Group 2 (Avg: 2.13) - 6 students:
  - Yara Knight (1.1)
  - Umar Harris (1.5)
  - Quinn Park (1.9)
  - Nancy Cruz (2.2)
  - Jackie Cohen (2.6)
  - George Brown (3.5)

Group 3 (Avg: 2.10) - 5 students:
  - Winston Browne (1.3)
  - Sam Lee (1.7)
  - Paulina Carter (2.0)
  - Leo Martin (2.4)
  - Henry Rodriguez (3.1)

Group 4 (Avg: 2.12) - 5 students:
  - Xander Evans (1.2)
  - Tina Lee (1.6)
  - Olga Allen (2.1)
  - Katherine Green (2.5)
  - John (3.19)
```

- **Explanation:** In this test case, the number of students doesn't divide evenly by the number of groups. The algorithm handles this by distributing the students across groups and ensuring that the groups are as balanced as possible.

The algorithm begins by creating a list of empty sublists. Each sublist represents a group, and the total number of sublists is equal to the number of groups.

Step 1: Initialize Groups

The algorithm begins by creating a list of empty sublists, where each sublist represents a group. The number of sublists equals the total number of groups.

Example for 4 groups:

groups = [[], [], [], []]

Step 2: Determine Base Students per Group & Remainder

To distribute students as evenly as possible:

Calculate the base number of students per group using integer division: students // groups.

Calculate the remainder using the modulo operator: students % groups. The remainder indicates how many groups will receive one extra student.

Example: For 22 students and 4 groups:

Base students per group = 22 // 4 = 5

Remainder = 22 % 4 = 2

Therefore, each group will have at least 5 students, and the first two groups will have 6 students to account for the remainder.

Step 3: Round-Robin Assignment

Students are assigned to groups in a round-robin fashion. Starting with the first group, each student is placed in the next group in sequence. After assigning a student to the last group, the algorithm loops back to the first group. The first remainder groups receive one additional student to distribute the extra students.

Example Assignment:

Given 22 students with their respective grades, the round-robin assignment results in:

Group 1: Zara Hill (1.05), Vera King (1.40), Rachel Wong (1.80), Nancy Cruz (2.20), Jackie Cohen (2.60), Roginand (3.23)

Group 2: Yara Knight (1.10), Umar Harris (1.50), Quinn Park (1.90), Mila Foster (2.30), Isaac Moore (3.00), John (3.19)

Group 3: Xander Evans (1.20), Tina Lee (1.60), Paulina Carter (2.00), Leo Martin (2.40), Henry Rodriguez (3.10)

Group 4: Winston Browne (1.30), Sam Lee (1.70), Olga Allen (2.10), Katherine Green (2.50), George Brown (3.50)

Step 4: Calculate Initial Group Averages

After the round-robin assignment, compute the average grade for each group to evaluate the balance.

Group Averages:

Group 1: (1.05 + 1.40 + 1.80 + 2.20 + 2.60 + 3.23) / 6 = 12.28 / 6 = 2.04

Group 2: (1.10 + 1.50 + 1.90 + 2.30 + 3.00 + 3.19) / 6 = 12.99 / 6 = 2.16

Group 3: (1.20 + 1.60 + 2.00 + 2.40 + 3.10) / 5 = 10.30 / 5 = 2.06

Group 4: (1.30 + 1.70 + 2.10 + 2.50 + 3.50) / 5 = 11.10 / 5 = 2.22

The initial averages show significant imbalance, with Group 4 having the highest average (2.964) and Group 1 the lowest (1.908).

Step 5: Balancing the Groups (Divide-and-Conquer)

To balance group averages, a divide-and-conquer approach recursively splits groups, swaps students to minimize differences, and merges subsets for refinement.

Divide

Groups are split into halves until each subset has one group.

Example for 4 groups:

Initial list: [Group 1, Group 2, Group 3, Group 4]

First split: [Group 1, Group 2] and [Group 3, Group 4]

Second split: [Group 1] and [Group 2] and [Group 3] and [Group 4]

Conquer

For each pair:

Calculate each group's average.

Identify the highest and lowest averages.

Swap students to reduce the difference, executing only if it improves balance.

Example Swap:

Group 1: Zara Hill (1.05), Vera King (1.40), Rachel Wong (1.80), Nancy Cruz (2.20), Jackie Cohen (2.60), Roginand (3.23)

Average: 2.047

Group 2: Yara Knight (1.10), Umar Harris (1.50), Quinn Park (1.90), Mila Foster (2.30), Isaac Moore (3.00), John (3.19)

Average: 2.165 Initial difference: 0.118 Swap Roginand (3.23) with Isaac Moore (3.00):

New Group 1: Average ≈ 2.008

New Group 2: Average ≈ 2.203 New difference: 0.195 (swap not executed). Try Group 1 and Group 4:

Group 4: Winston Browne (1.30), Sam Lee (1.70), Olga Allen (2.10), Katherine Green (2.50), Henry Rodriguez (3.10)

Average: 2.140 Swap Roginand (3.23) with Henry Rodriguez (3.10):

New Group 1: Average ≈ 2.025

New Group 4: Average ≈ 2.166 New difference: 0.141 (swap executed).

Merge

Merge subsets back: [Group 1, Group 2, Group 3, Group 4]. Repeat conquering until no swaps improve balance.

Updated averages after swap:

Group 1: 2.025

Group 2: 2.165

Group 3: 2.140

Group 4: 2.166

The algorithm iterates until no further improvement is possibl**e.**

Step 6: Final Group Assignment

After multiple iterations of the divide-and-conquer phase, the algorithm converges on a balanced assignment with minimal differences in average grades.

Final Groups:

Group 1: Zara Hill (1.05), Vera King (1.40), Rachel Wong (1.80), Mila Foster (2.30), Isaac Moore (3.00), Roginand (3.23)

Average: (1.05 + 1.40 + 1.80 + 2.30 + 3.00 + 3.23) / 6 = 12.78 / 6 = 2.13

Group 2: Yara Knight (1.10), Umar Harris (1.50), Quinn Park (1.90), Nancy Cruz (2.20), Jackie Cohen (2.60), George Brown (3.50)

Average: (1.10 + 1.50 + 1.90 + 2.20 + 2.60 + 3.50) / 6 = 12.80 / 6 = 2.13

Group 3: Winston Browne (1.30), Sam Lee (1.70), Paulina Carter (2.00), Leo Martin (2.40), Henry Rodriguez (3.10)

Average: (1.30 + 1.70 + 2.00 + 2.40 + 3.10) / 5 = 10.50 / 5 = 2.10

Group 4: Xander Evans (1.20), Tina Lee (1.60), Olga Allen (2.10), Katherine Green (2.50), John (3.19)

Average: (1.20 + 1.60 + 2.10 + 2.50 + 3.19) / 5 = 10.59 / 5 = 2.12

The final averages (2.130, 2.133, 2.100, 2.118) are closely balanced, with a maximum difference of approximately 0.033, indicating a successful balancing phase.

Complexity Analysis:

- **Time Complexity:**
  - **Let:**
    - **n = total number of students**
    - **g = number of groups**
    - **s = members of group**
    - **r = number of rebalancing rounds (in worst case)**
    - **g1 = first group**
    - **g2 = second group**
  - add(Student) = **O(log n)**
  - assignGroup(groupAmount) = **O(r * g * s)**
  - calculateGroupAverage(group) = **O(s)**
  - improveBySwapping (groups, group1index, group2 index)
    - Best Case: O(s)
    - Worst Case: O(s)
  - mergeAndBalance(groups, left, right)
    - Best case: O(g * s) where: (r = O(1))
    - Worst Case: O(r * g * s)
  - balanceGroupsDivideConquer(groups, 0, g)
    - Base Case: O(g log g * s) where: r = O(1)
    - Worst Case: O(g log g * r * s)
  - Overall
    - Best Case: O (g log g * s); assuming r= O(1)
    - Worst Case: O(g log g * r * s)
- **Space Complexity**
  - **TreeSet<Student>: O(n)**
  - **List<List<Student>>: O(n)**
  - **LinkedList<Student>: O(n)**
  - **Overall Space Complexity: O(n)**
- **Trade offs:**
  - **Time Complexity**: The performance of the algorithm depends on both the dataset size and the distribution of grades. For datasets with grades that are

already fairly balanced, the algorithm runs efficiently. However, as the number of students increases or if the grades are highly unbalanced, the algorithm may experience a drop in performance, particularly in the balancing and swapping steps, which could approach O(g log g * r * s) in the worst case.

- **Space Complexity**: The space usage scales linearly with the number of students, O(N). Since all the students and groups are stored in a TreeSet and lists, the space complexity is manageable for smaller datasets. However, for very large datasets, the space requirement could become a constraint.

Reflection

At first, I wanted to create a new data structure from scratch—specifically, a faster version of a linked list. But after some research, I realized that my idea already existed. So I shifted my focus toward building a practical algorithm by combining existing techniques. I initially approached the group assignment problem in a naive way. But as I continued working on the algorithm, I developed a deeper appreciation for well-known algorithms and how effective they are. What started as a simple distribution task evolved into a more thoughtful design that balanced simplicity and sophistication. I began with basic sorting and group distribution. However, to achieve fairer results, I had to integrate divide-and-conquer. This recursive approach was inspired by algorithms like merge sort, which is great at handling complex problems by breaking them down.

The balancing phase was the most difficult part. Designing a swap system that actually improved balance without breaking group size constraints took time. Debugging edge cases, especially with uneven group sizes, tested my patience and problem-solving skills. But it was satisfying to see the algorithm eventually balance the groups with near-equal averages.

The algorithm could be improved by using dynamic programming to avoid redundancy of numbers of swap by caching it, but that would add a lot of complexity, which wasn't practical for now due to time constraints, and using the divide-and-conquer approach added complexity to the current implementation.

Overall, this project deepened my understanding of algorithmic design and how existing paradigms can be adapted for real problems. It also showed the value of iterative refinement. Combining round-robin, greedy partitioning, and divide-and-conquer turned a simple idea into a strong solution. In the future, I'd like to explore parallel processing for the balancing phase to make it scalable for large datasets.